

软件过程改进方案设计文档

学号：2023141461061

姓名：袁新坪

项目名称：AI辅助软件过程改进平台

文档版本：V1.0

编写日期：2026年7月

摘要

本文针对课程项目中需求不清晰、任务状态不透明、代码评审不充分、测试介入较晚、缺陷重复出现和项目经验难以沉淀等问题，设计一套轻量级软件过程改进方案。方案以“过程可视、质量前移、自动检查、数据驱动、持续复盘”为核心原则，将需求、设计、开发、评审、测试、发布和改进活动连接成闭环。

方案采用分阶段实施方式。第一阶段统一流程与模板，第二阶段建立自动化质量门禁，第三阶段引入AI辅助需求检查、代码评审、测试生成和知识问答，第四阶段通过过程指标和迭代复盘持续优化。文档给出了改进目标、现状分析、目标流程、角色职责、质量门禁、度量指标、实施计划、风险控制和验收标准。

关键词： 软件过程改进；质量保证；持续集成；代码评审；AI辅助；过程度量

1. 文档目的

本方案用于指导“AI辅助软件过程改进平台”课程项目的软件过程改进活动，具体目标为：

- 建立统一、可执行的软件开发流程；
- 降低需求歧义和需求变更带来的返工；
- 提高代码评审和测试活动的有效性；
- 建立从需求到代码、测试和缺陷的追踪关系；
- 引入自动化工具和AI辅助能力；
- 使用过程数据评价改进效果；
- 形成可复用的项目规范和经验库。

2. 现状分析

2.1 当前过程描述

项目当前采用迭代开发方式，主要活动包括需求讨论、任务分配、编码、简单测试和版本提交。流程能够支持基本开发，但缺少明确入口条件、出口条件和质量门禁。

2.2 主要问题

编号	问题	影响
P01	需求描述格式不统一	开发理解不一致，验收困难
P02	任务拆分粒度不稳定	进度难估算，任务延期不易发现
P03	代码提交信息不规范	难以追溯变更目的
P04	代码评审依赖自觉	缺陷较晚暴露
P05	测试在开发后期集中进行	修复成本高
P06	缺陷缺少分类和根因分析	同类问题重复出现
P07	发布检查依靠人工记忆	容易遗漏配置和脚本
P08	缺少过程指标	改进效果无法量化
P09	文档和经验分散	新成员学习成本高
P10	AI工具使用无统一规范	存在隐私和错误采用风险

2.3 根因分析

采用“人员、流程、技术、数据、管理”五个维度进行分析：

- **人员：** 团队经验差异较大，质量责任边界不清晰；
- **流程：** 活动存在但缺少标准输入输出；
- **技术：** 自动测试、静态检查和持续集成不足；
- **数据：** 需求、提交、测试和缺陷之间未建立关联；
- **管理：** 只关注是否按时完成，缺少质量和过程指标。

3. 改进目标

3.1 总体目标

建立一个轻量、可执行、可度量的软件过程，使团队能够以较低管理成本稳定完成迭代开发，并通过自动化和AI减少重复劳动。

3.2 量化目标

以下指标作为课程项目的目标值，实际执行时可根据项目规模调整：

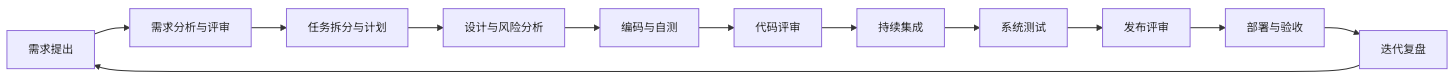
指标	当前基线	目标
需求包含验收标准的比例	约40%	不低于90%
合并请求评审覆盖率	约50%	100%
自动化构建成功率	未稳定统计	不低于95%
关键模块单元测试覆盖	较低	不低于70%
缺陷重开率	未统计	低于10%
高优先级缺陷平均修复时间	2—3天	不超过1天
需求到测试用例追踪率	较低	不低于90%
发布检查项完成率	依赖人工	100%
迭代复盘执行率	不固定	100%

4. 改进原则

- 1. **轻量化：** 流程服务于交付，不为形式增加无效文档。
- 2. **质量前移：** 在需求和设计阶段尽早发现问题。
- 3. **自动优先：** 可由工具稳定检查的内容尽量自动化。
- 4. **人员负责：** AI和工具只提供建议，最终责任由角色承担。
- 5. **全程追踪：** 需求、任务、提交、测试和缺陷建立关联。
- 6. **数据驱动：** 使用指标评价问题和改进效果。
- 7. **持续优化：** 每个迭代选择少量重点问题改进。

5. 目标软件过程

5.1 总体流程



5.2 需求阶段

输入

- 用户需求或问题描述；
- 业务背景；
- 原型或参考资料；
- 约束条件。

活动

- 编写用户故事；
- 明确业务规则；
- 定义验收标准；
- 分析异常和边界场景；
- 评估优先级和工作量；
- 使用AI进行歧义检查。

输出

- 评审通过的需求条目；
- 验收标准；
- 初步测试场景；
- 风险与依赖清单。

入口和出口标准

类型	条件
入口条件	需求来源明确、背景可说明
出口条件	描述清晰、验收标准可测试、优先级确定、依赖已识别

5.3 计划与任务拆分阶段

需求应拆分为可在较短时间内完成的任务。每个任务必须包含：

- 关联需求编号；
- 负责人；
- 预计工作量；
- 完成定义；
- 依赖关系；
- 风险说明；
- 测试要求。

任务不应使用“完成后端”“处理页面”等过于宽泛的描述，而应拆分为可验证的工作单元。

5.4 设计阶段

对于涉及多个模块、数据库变更、权限、安全、外部接口或性能风险的需求，必须进行简要设计。

设计内容至少包括：

- 模块边界；
- 数据模型变更；
- 接口输入输出；
- 错误处理；
- 权限控制；
- 兼容性；
- 测试策略；
- 回滚方案。

5.5 编码阶段

分支策略

- `main`：稳定版本；
- `develop`：当前集成版本；
- `feature/*`：功能开发；
- `fix/*`：缺陷修复；
- `release/*`：发布准备。

提交规范

建议格式：

类型(模块)：简要说明

关联需求：REQ-xxx

关联任务：TASK-xxx

类型可使用 feat 、 fix 、 test 、 docs 、 refactor 、 build 等。

完成定义

代码进入评审前必须满足：

- 本地编译通过；
- 已完成必要自测；
- 不包含敏感数据；
- 关键逻辑有注释；
- 关联需求和任务；
- 新增功能包含相应测试；
- 无明显静态检查错误。

5.6 代码评审阶段

每个合并请求至少由一名非作者成员评审。重要模块可要求两人评审。

评审内容包括：

类别	检查内容
正确性	是否满足需求和业务规则
可读性	命名、结构和注释是否清晰
可维护性	是否存在重复、过度耦合和复杂逻辑
安全性	权限、输入校验、敏感数据处理
性能	循环、查询、缓存和资源使用
兼容性	是否影响已有接口和数据
测试	是否有足够测试及边界场景

类别	检查内容
文档	接口或配置是否同步更新

AI代码评审可在人工评审前自动执行，用于发现常规问题，但不能代替人工批准。

5.7 持续集成阶段

每次提交或合并请求触发以下检查：

1. 依赖安装；
2. 编译或构建；
3. 代码格式检查；
4. 静态分析；
5. 单元测试；
6. 接口测试；
7. 安全扫描；
8. 生成构建报告；
9. 记录质量门禁结果。

任何高严重度问题、编译失败或关键测试失败都应阻止合并。

5.8 测试阶段

测试活动从需求阶段开始，而非编码结束后开始。

测试层次包括：

- 单元测试；
- 接口测试；
- 集成测试；
- 系统测试；
- 回归测试；
- 安全测试；
- 用户验收测试。

缺陷提交必须包含：

- 缺陷标题；
- 发现版本；
- 环境信息；

- 前置条件；
- 重现步骤；
- 实际结果；
- 期望结果；
- 严重程度；
- 截图或日志；
- 关联需求。

5.9 发布阶段

发布前执行检查清单：

- 需求全部达到验收状态；
- 高优先级缺陷已关闭；
- 自动化测试通过；
- 数据库脚本已评审；
- 配置差异已确认；
- 回滚方案可执行；
- 版本号和变更记录已更新；
- 备份已完成；
- 发布负责人和验证负责人明确。

5.10 复盘阶段

每个迭代结束后进行复盘，讨论：

- 哪些做法有效；
- 哪些问题影响交付；
- 哪些缺陷本可提前发现；
- 哪些自动化规则需要增加；
- AI建议的采用率和误报情况；
- 下一迭代最多选择三项改进措施。

6. 角色与职责

角色	主要职责
项目负责人	计划、风险、资源协调和改进决策
产品/需求负责人	需求澄清、优先级和验收
开发负责人	技术方案、代码质量和集成
开发人员	编码、自测、文档和缺陷修复
测试人员	测试设计、执行、缺陷跟踪和质量报告
质量保证人员	流程检查、指标收集和改进跟踪
配置管理员	分支、版本、构建和发布管理
AI管理员	模型配置、数据脱敏、提示模板和使用审计

7. AI辅助机制设计

7.1 需求检查

AI对需求进行以下检查：

- 模糊词；
- 未定义对象；
- 缺少前置条件；
- 缺少异常处理；
- 不可验证要求；
- 验收标准不足；
- 与历史需求冲突。

7.2 代码评审

AI对合并请求生成结构化报告，每条建议包含：

- 文件和位置；
- 问题类型；

- 严重程度；
- 问题解释；
- 修改建议；
- 是否需要人工确认。

7.3 测试生成

AI根据需求和接口定义生成测试用例初稿。测试人员必须筛选、修改和确认，系统记录“生成、采纳、修改、废弃”状态，用于评价AI质量。

7.4 知识沉淀

已确认的需求规则、设计决策、缺陷根因和发布经验进入知识库。未确认的AI内容不得直接进入正式知识库。

8. 过程度量设计

8.1 进度指标

- 迭代完成率；
- 需求交付周期；
- 任务延期率；
- 在制任务数量；
- 代码评审等待时间。

8.2 质量指标

- 缺陷数量和严重度分布；
- 缺陷密度；
- 缺陷重开率；
- 自动化测试通过率；
- 构建成功率；
- 线上缺陷比例。

8.3 改进指标

- AI建议采纳率；

- AI误报率；
- 需求问题提前发现数量；
- 自动生成测试用例有效率；
- 平均文档编写时间变化；
- 因流程遗漏导致的返工次数。

9. 实施计划

阶段	周期	主要工作	交付物
第一阶段	第1周	现状调研、模板统一、角色确认	现状报告、模板
第二阶段	第2周	建立需求、评审、测试和发布流程	流程定义、检查表
第三阶段	第3周	接入持续集成和自动检查	CI脚本、质量门禁
第四阶段	第4周	引入AI需求检查和测试生成	AI原型、验证记录
第五阶段	后续迭代	指标分析和流程优化	度量报告、改进清单

10. 风险与应对

风险	影响	应对措施
团队认为流程增加负担	执行率低	控制模板长度，自动生成重复内容
自动检查误报较多	团队忽略工具	分级展示，持续调整规则
AI输出不准确	错误决策	强制人工确认，保存来源
项目数据不完整	指标失真	统一字段和状态，定期检查
工具集成复杂	延迟实施	优先使用仓库已有能力
过度追求指标	形式主义	指标与实际质量结果结合
敏感信息泄露	安全问题	脱敏、权限控制、私有化选项

11. 验收标准

本方案实施完成后应满足：

1. 团队成员可以说明主要开发流程和角色职责；
2. 每个需求均有统一模板和验收标准；
3. 每个合并请求都有评审记录；
4. 自动构建和测试可以在提交后触发；
5. 发布前有完整检查记录；
6. 需求、任务、提交、测试和缺陷可建立关联；
7. 至少两个AI辅助场景完成验证；
8. 项目能够输出基础进度和质量指标；
9. 每个迭代形成复盘和改进清单；
10. 流程执行记录可以追溯。

12. 结论

本方案通过统一流程、质量门禁、自动化工具、AI辅助和过程度量，将原本依赖个人经验的软件开发活动转变为可执行、可检查和可持续改进的过程。方案强调轻量化和渐进实施，避免在课程项目中引入过重管理成本。

AI技术在本方案中主要承担需求检查、代码评审建议、测试用例生成和项目知识检索等辅助任务。所有关键结论均需人员确认。通过持续记录AI建议和采用结果，可进一步评价其实际价值，并为后续过程优化提供数据。

附录A：需求检查清单

- ☐ 需求背景是否明确
- ☐ 用户和使用场景是否明确
- ☐ 输入、处理和输出是否说明
- ☐ 业务规则是否完整
- ☐ 异常场景是否覆盖
- ☐ 权限要求是否明确
- ☐ 性能要求是否可度量
- ☐ 验收标准是否可测试

- ☐ 是否识别依赖和风险
- ☐ 是否关联测试场景

附录B：合并请求检查清单

- ☐ 代码关联需求和任务
- ☐ 本地构建通过
- ☐ 已完成自测
- ☐ 新增或修改测试
- ☐ 无敏感信息
- ☐ 数据库脚本已评审
- ☐ 接口文档已更新
- ☐ AI评审建议已处理
- ☐ 人工评审已通过
- ☐ 持续集成全部通过